

Step-1 — Edge-Board CNN Characterization Report

Aetina RKC-A02 (RK3588 + Axelera Metis Alpha) · 5 CNN workloads × 3 compute units · Phase 1a (vendor pipelines, 45 cells) + Phase 1b (direct-tensor harness with batch sweep, 180 cells)

Reading guide. Every term that might be unfamiliar is defined in the “Key terms” section below. Sections default to expanded — click any heading bar with a ▼ marker to collapse it. The PDF version renders everything expanded.

Executive summary

The board is the **Aetina RKC-A02**: a Rockchip RK3588 SoC carrier with three usable inference accelerators — an Axelera **Metis Alpha** 214-TOPS AIPU on PCIe, the SoC's built-in **RKNPU2** 6-TOPS NPU, and a **Mali-G610** GPU. We measured 5 representative CNN workloads across all three units, first through each vendor's stock inference path (Phase 1a), then through a direct-tensor harness that bypasses host-side pipeline overhead and sweeps batch size (Phase 1b).

Single most important finding: **the bottleneck depends entirely on how you feed the unit**. The same Metis chip on the same model returns 174 FPS through the vendor pipeline and 1144 FPS through a direct-tensor 4-instance harness — a **6.6×** swing on the same hardware. Phase 1a's “flat throughput across 38× compute” result was a real measurement of the wrong thing: it characterised the host CPU's preprocessing pipeline, not the AIPU. Once you bypass the pipeline, the AIPU's throughput scales with model compute as you'd expect.

The three units, fed properly, show three different bottleneck regimes:

- **Metis** — host-pipeline-bound through the vendor stack; *compute-bound* once fed directly. Multi-instance multi-threading fills all 4 AIPU cores. True single-instance batching is mostly blocked by a Voyager SDK reshape limitation (13/15 batched compiles failed). 1144 FPS MobileNetV2 is the headline.
- **RKNPU2** — *latency-bound* at batch=1, *throughput-bound* under batching. Batching scales throughput 5–10× per model. At batch=32 it beats Phase-1a Metis on every classification model (ResNet-152: 472 FPS vs 173).

- **Mali** — *memory-bandwidth-saturated*. Batching adds almost nothing; big batched models exceed the 15-min/200-loop benchmark timeout. Not a viable batched-inference unit, but the realistic *preprocessing* accelerator on this board (where Phase 1a's bottleneck actually lives).

Key terms (read this first)

▼ Glossary — FPS, latency, GMACs, batch, vendor pipeline, host-pipeline-bound, colorconvert, DMA, multi-instance, INT8 / FP16, ...

FPS (Frames Per Second)

Throughput — how many inferences the unit completes each second. **Higher is better**. If a unit reports 100 FPS for one image at a time, you can process 100 images per second sustained.

Computed as $\text{FPS} = 1000 / \text{median_latency_ms}$ at batch=1, or $\text{FPS} = 1000 \cdot \text{batch} / \text{median_per-call_latency_ms}$ when each device call processes a batch of frames.

Latency

How long one inference takes, in milliseconds. **Lower is better**. FPS is throughput; latency is the per-frame wait. For batch>1, per-call latency rises but per-frame latency typically falls.

Batch size

How many inputs the device processes in a single call. Batch=1 means one image at a time (the real-time camera case). Batch=32 means feed 32 images, get 32 results back from one device call (the offline / video-file case). Batching usually raises throughput because fixed per-call overheads are amortised across many frames, but it also raises per-call latency, uses more memory, and requires the model to be compiled for that batch dimension.

Parameters

The numeric weights in the model that are learned during training. Measured in millions (M). In INT8 each parameter occupies 1 byte, so a 25 M-parameter model takes about 25 MB of weight memory.

MAC / GMAC

MAC = Multiply-Accumulate, one fused multiply-add operation ($a \times b + c$). It is the dominant primitive in neural network compute (convolutions, fully connected layers, attention all sum products). GMAC = 10^9 MACs **per inference**. Roughly, FLOPs $\approx 2 \times$ MACs (each MAC is one multiply plus one add). It is a model-property number — intrinsic to the model and the input shape,

independent of which hardware runs it. Higher GMAC = more compute per inference, which on a compute-bound unit means lower FPS.

INT8 / FP16 / FP32

Numeric precision used during inference. FP32 (32-bit float) is the training format. FP16 (16-bit float) is half-precision, used by GPU-style units. INT8 (8-bit integer) is the compressed quantised format used by dedicated NPUs and AIPUs — 4× smaller weights and faster per-MAC, with a small accuracy cost from quantisation. Different units run at different native precisions: on this board Metis and RKNPU2 run INT8, Mali runs FP16.

Vendor pipeline

The manufacturer-supplied complete inference application. For Axelera Metis this is `inference.py`, which builds a full GStreamer (video processing framework) pipeline: *video source* → *format conversion* → *resize* → *normalisation* → *device inference* → *post-processing*. It is the easy / officially-supported way to run inference, but it adds a lot of per-frame work on the host CPU. The Phase 1a numbers were measured through this pipeline; the Phase 1b numbers bypass it by calling the device runtime API directly with pre-prepared tensors.

Host-pipeline-bound

A bottleneck verdict: the throughput ceiling is set by host (CPU) work in the inference pipeline, not by the accelerator. When a model's accelerator-compute step takes 5 ms but the host's preprocessing takes 15 ms per frame, you'll get ~65 FPS no matter how much faster the accelerator is. Phase 1a found this is the case for Metis through the vendor pipeline: throughput is flat at ~170 FPS across a 38× compute range, because every model is gated by the same CPU preprocessing step.

Colorconvert

Converting pixel format. Cameras and video decoders typically output frames as YUV (a video-friendly luminance + chroma layout) or BGR (OpenCV default). Neural networks usually want RGB, often normalised to specific mean/std values. Converting between formats is a per-pixel CPU operation. On the Voyager pipeline this is the GStreamer element `axtransform-colorconvert0`; at a 640×480 frame size it ate ~14–21 ms per frame in Phase 1a, capping the pipeline at ~50–80 effective FPS for that element alone.

DMA (Direct Memory Access)

Hardware-assisted block memory copy between host RAM and a device (the Metis card here, over PCIe). DMA is fast but each transfer has fixed setup cost (the driver programs the DMA engine, waits for completion, etc.). On this board we run Metis with `dma_poll=1` (polling-mode DMA — a workaround for an interrupt-delivery incompatibility on this carrier; see the platform baseline

notes), which makes each DMA round-trip a fixed cost on the order of a few milliseconds. **Fixed per-inference DMA overhead** means that cost is paid for every device call regardless of how much compute the call does — so a small model (50 μ s of real AIPU work) ends up dominated by 5 ms of DMA round-trip, while a big model amortises it better. Batching is the standard way to amortise this: do 32 frames in one call, pay the DMA setup once.

Multi-instance / 4-instance multi-threading

A way to fill all cores of a multi-core accelerator without changing the model. The Metis AIPU has 4 cores. A single batch=1 model instance occupies 1 core; the other 3 sit idle. With *4-instance multi-threading* you load 4 independent inference contexts (one per core) and dispatch from 4 host threads, so all 4 AIPU cores are working on different frames in parallel. Different from batching (which packs more frames into one call), but the throughput effect on Metis is similar: $\sim 4\times$ over single-instance. Phase 1b uses this to get the headline 1144 FPS for MobileNetV2.

PCIe Gen3 $\times 4$

The bus between host and Metis card. Theoretical peak ~ 3.94 GB/s per direction ($8 \text{ GT/s} \times 4 \text{ lanes} \times 128\text{b}/130\text{b}$ coding). Effective DMA throughput ~ 3.5 GB/s in practice. Every input tensor goes host $\rightarrow$ card, every output tensor goes card $\rightarrow$ host, over this bus.

Quantisation calibration

The step that converts FP32 weights and activations to INT8. Each tensor needs a scale and zero-point chosen so that the int8 range covers the typical activation distribution. Calibration runs a few hundred representative input images through the model, observes the activation ranges, and picks the scales. All three units on this board use the same COCO-derived 200-image representative set for calibration parity.

Hardware

▼ The three compute units and the memory hierarchy

The three compute units

Unit	Type	Peak compute	Memory	Bus to host
Metis Alpha (PCIe card)	4 AIPU cores, each with a 512×512 in-memory matrix-vector unit, a depthwise unit, and an LUT for activations.	~214 TOPS INT8	4 MB L1 scratchpad per AIPU core (activations); 32 MB on-card L2 SRAM (weights); no on-card DDR (it's the <i>Alpha</i> silicon; production Metis has on-card DDR).	PCIe Gen3 ×4, ~3.5 GB/s effective DMA
RKNPU2 (on-SoC)	Rockchip RK3588's built-in 3-core neural network accelerator.	6 TOPS INT8 @ 1.0 GHz	Shares the SoC's 16 GB LPDDR4 (no separate accelerator memory).	On-chip AXI fabric (no PCIe in path)
Mali-G610 MP4 (on-SoC)	Arm 3D graphics GPU with 4 shader cores; supports OpenCL 3.0 / Vulkan 1.2. Designed for graphics first; INT8 NN compute is an afterthought.	(Not advertised for ML; FP16 useful, INT8 immature)	Shares the SoC's 16 GB LPDDR4.	On-chip

Memory hierarchy (fastest → slowest)

Tier	Size	Who can read	Notes
Metis L1 SPM	4 MB per AIPU core (16 MB total)	One AIPU core	Activations scratchpad; single-cycle access
Metis L2 SRAM	32 MB shared	All 4 AIPU cores	Weights home (the compiled <code>pool_l2_const</code>); models >32 MB INT8 must spill weights to host DDR
Host LPDDR4	16 GB	CPU / RKNPU2 / Mali / Metis (via PCIe)	~40–50 GB/s for CPU; Metis sees it through the PCIe DMA window (~3.5 GB/s)
eMMC	14 GB root + 15 GB <code>/userdata</code>	CPU	Model files, build artefacts

Why these numbers matter for the results

Several findings in this report trace directly to the hardware table:

- **Metis's 32 MB on-card L2 SRAM** is the cliff every big batched model hits. ResNet-152's 60 MB INT8 weights already exceed L2 at batch=1, so the compiler spills weights to host DDR over PCIe. Batched activations on top of this fill the budget further.
- **PCIe Gen3 x4 at ~3.5 GB/s** is the “DMA fixed cost” mentioned throughout. Combined with the `dma_poll=1` polling-mode workaround on this carrier, every Metis device call has a few-millisecond round-trip floor. That floor is what dominates the AIPU's `inference` element time at batch=1 in Phase 1a (~5–8 ms regardless of model).
- **RKNPU2 and Mali share LPDDR4 with the host.** No PCIe to cross, no fixed transfer overhead — explaining why RKNPU2 scales beautifully with batching (no per-call DMA round-trip to amortise) and Mali saturates LPDDR bandwidth quickly on bigger batches.

The 5 workload models

▼ Detailed reference card per model (architecture, params, GMACs, why it's in the set)

The benchmark deliberately spans a 38× range of compute (0.30 → 11.51 GMACs) and three architecture families (mobile/depthwise, classical convnet, detection). Each model exposes a different aspect of the hardware.

MobileNetV2

Image classification — ImageNet, 1000 classes · introduced 2018 (Sandler et al., CVPR 2018)

Input shape 224×224 RGB

Parameters 3.49 M (≈ 3 MB at INT8)

Compute per
inference 0.301 GMACs

FP32 reference
accuracy ImageNet top-1 ≈ 71.8%

Architecture	Inverted-residual blocks with linear bottlenecks and depthwise-separable convolutions; ReLU6 activations.
Why it's in this set	Smallest/lightest CNN — the mobile-architecture baseline; stresses depthwise-conv kernel efficiency.
Quirks that matter for benchmarking	Depthwise convolutions have very low arithmetic intensity (few MACs per byte of weight), so they often look memory-bound rather than compute-bound. NPU/GPU INT8 kernels for depthwise convolutions vary widely in quality.

EfficientNet-B0

Image classification — ImageNet, 1000 classes · introduced 2019 (Tan & Le, ICML 2019)

Input shape	224×224 RGB
Parameters	5.27 M ($\approx$ 5 MB at INT8)
Compute per inference	0.386 GMACs
FP32 reference accuracy	ImageNet top-1 $\approx$ 77.3%
Architecture	Compound-scaled mobile architecture (NAS-derived); MBConv blocks (mobile inverted bottlenecks) with squeeze-and-excitation; Swish (SiLU) activations.
Why it's in this set	Higher accuracy at the same scale as MobileNetV2 — tests how well each unit handles SE blocks and Swish.
Quirks that matter for benchmarking	Swish ($x \cdot \text{sigmoid}(x)$) is non-piecewise and hard to quantise cleanly in INT8. SE blocks insert global-pool $\rightarrow$ FC $\rightarrow$ broadcast-multiply, which can stall accelerators tuned for straight convolutional flow.

ResNet-50

Image classification — ImageNet, 1000 classes · introduced 2015 (He et al., CVPR 2016)

Input shape 224×224 RGB

Parameters	25.53 M ($\approx$ 25 MB at INT8)
Compute per inference	4.089 GMACs
FP32 reference accuracy	ImageNet top-1 $\approx$ 76.0%
Architecture	Classical deep residual network; bottleneck blocks ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$ convolutions with skip connection); ReLU activations.
Why it's in this set	The reference convnet — well-tuned across every inference stack; a 10 $\times$ compute step up from the mobile models.
Quirks that matter for benchmarking	Regular dense convolutions with high arithmetic intensity — compute-bound on most accelerators. Straightforward to quantise. BatchNorm folds cleanly into convs.

YOLOv8n (nano)

Object detection — COCO, 80 classes · introduced 2023 (Ultralytics)

Input shape	640 $\times$ 640 RGB
Parameters	3.15 M ($\approx$ 3 MB at INT8)
Compute per inference	4.372 GMACs
FP32 reference accuracy	COCO mAP $\approx$ 37%
Architecture	CSPDarknet backbone + PAN-FPN neck + decoupled detection head; SiLU activations; small (nano) variant of the YOLOv8 family.
Why it's in this set	The only detection task in the matrix, and the only model at 640 $\times$ 640 input — 8.2 $\times$ more input pixels than the classifiers.
Quirks that matter for benchmarking	Multi-scale FPN outputs and a detection head with concat / sigmoid / anchor decoding. High-resolution input dominates memory bandwidth on GPU-class units. Fewest parameters in the set but high GMACs because of the input resolution.

ResNet-152

Image classification — ImageNet, 1000 classes · introduced 2015 (He et al., CVPR 2016)

Input shape	224×224 RGB
Parameters	60.12 M ($\approx$ 60 MB at INT8)
Compute per inference	11.514 GMACs
FP32 reference accuracy	ImageNet top-1 $\approx$ 78.3%
Architecture	Same residual-bottleneck building block as ResNet-50, just 3× deeper (152 layers); ReLU activations.
Why it's in this set	The heavyweight compute / weight-memory anchor of the matrix — 38× the GMACs of MobileNetV2 and 17× the parameters.
Quirks that matter for benchmarking	Depth makes per-frame latency long even at high compute throughput. Weight memory (60 M $\times$ 1 byte INT8 $\approx$ 60 MB) exceeds the Metis Alpha's 32 MB on-card L2 SRAM, forcing the compiler to stream weights from host DDR over PCIe — a different memory regime to the smaller models.

Methodology

▼ How we measured (Phase 1a vs Phase 1b harnesses, protocol, precision policy, what's intentionally not measured)

Two measurement phases, deliberately different

The campaign runs two passes over the same 5 models $\times$ 3 units grid because they answer different questions and use different harnesses.

Phase 1a — vendor pipelines (batch=1, 45 cells)

- **Metis:** Voyager `inference.py --show-stats` . The full streaming GStreamer pipeline: video source (synthetic `fakevideo:640×480`) $\rightarrow$ colorconvert $\rightarrow$ Mali OpenCL resize $\rightarrow$ CPU pad/quantise $\rightarrow$ AIPU inference $\rightarrow$ CPU postprocess.

- **RKNPU2:** rknn-toolkit2's `rknn.inference()` on dummy input. No preprocessing in the measured path.
- **Mali:** MNN's `MNNV2Basic.out` OpenCL FP16 benchmark. Dummy input.

What this answers: *what does the realistic vendor-supplied path deliver?* Phase 1a uncovered the host-pipeline-bound regime on Metis.

Phase 1b — direct-tensor harness, batch sweep (180 cells)

- **Metis:** custom harness using the `axelera.runtime` Python API. Creates N model instances ($N = 4 / \text{batch}$) so all 4 AIPU cores are filled, dispatches from 4 host threads. Pure AIPU compute + DMA; no host preprocessing in the measured path.
- **RKNPU2:** `rknn.inference()` in a tight loop with batched dummy input.
- **Mali:** `MNNV2Basic.out` with batched `inputSize`.

What this answers: *once you remove the host pipeline, what does the unit itself deliver, and how does that change with batch?*

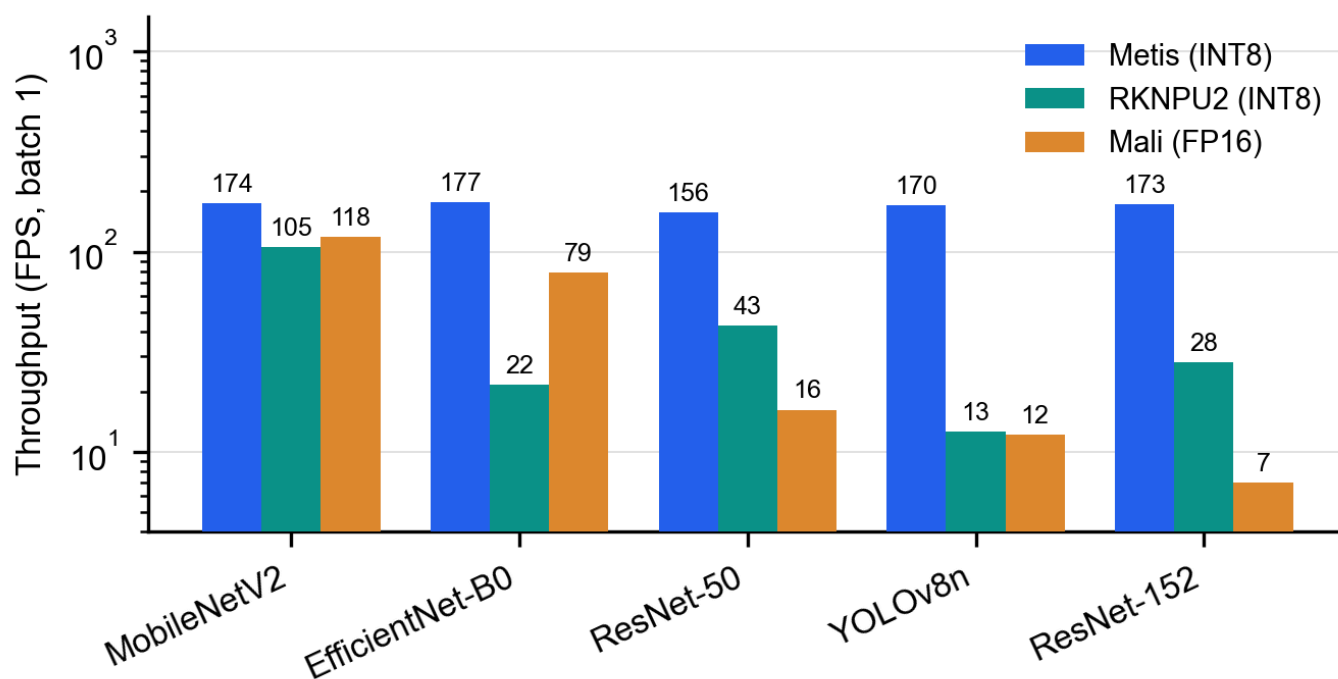
Common protocol

- Warm-up: 20 inferences discarded before measurement.
- Steady state: ≥ 300 inferences and ≥ 30 s of measurement, whichever takes longer.
- Three repeats per cell from separate cold-start processes.
- Per-unit native precision (Metis INT8, RKNPU2 INT8, Mali FP16). Mali FP16 is the realistic Mali deployment precision — true INT8 GEMM on a Mali GPU is not well supported in the MNN OpenCL backend.
- Shared calibration data (COCO 2017 representative subset, 200 images) across all three INT8 quantisation flows for parity.
- Power and energy are intentionally *not* reported: this board has no on-board power instrumentation (no INA, no current sensors). Bottleneck attribution — the actual objective — does not need power numbers.

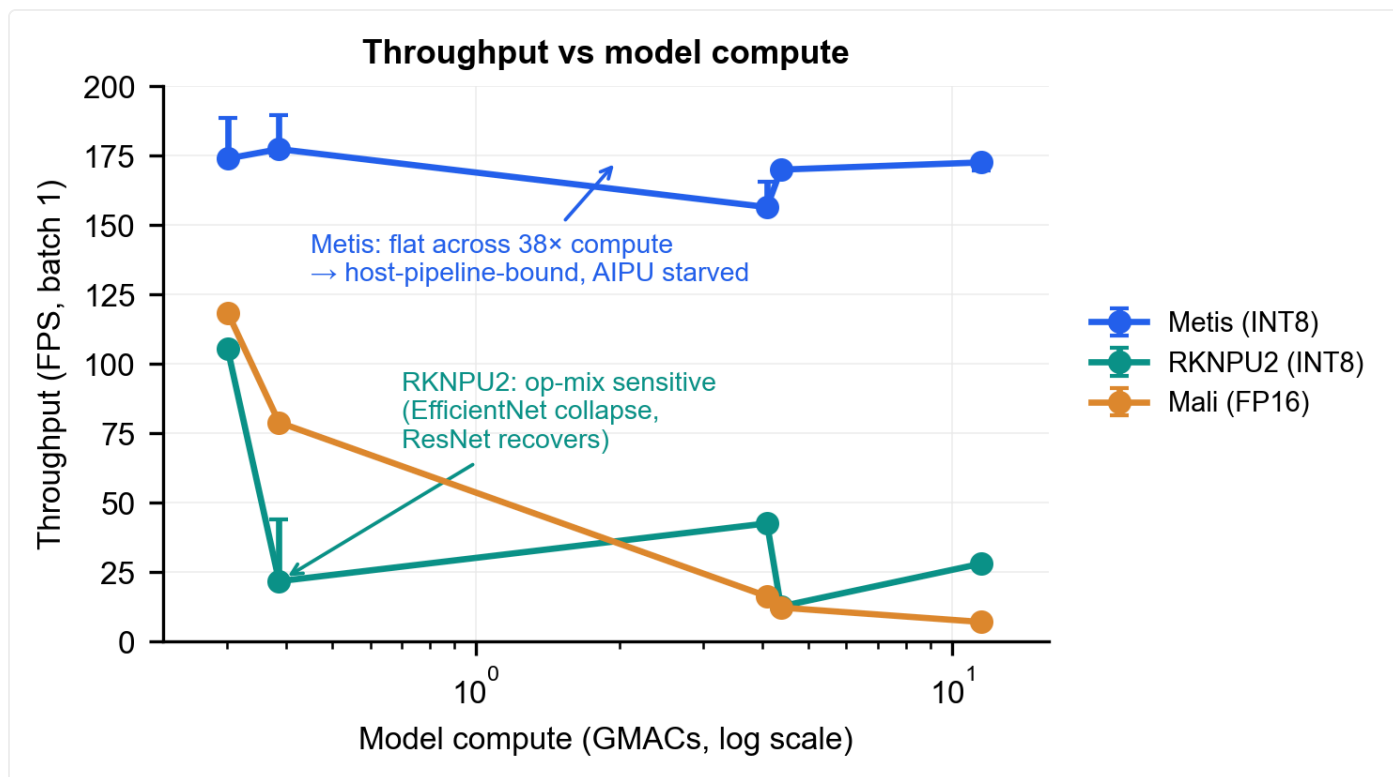
Phase 1a results — vendor pipelines, batch=1

Phase 1a measures the realistic vendor path. The result is striking: **Metis throughput is flat at 157–177 FPS across a 38× range of model compute**. A compute-bound unit would fall off a cliff as compute rises; Metis does not move.

Per-model throughput by compute unit



Per-model throughput by compute unit (Phase 1a, batch=1, vendor pipelines, log scale). Metis (blue) tops every model at 156–177 FPS regardless of compute. RKNPU2 (teal) and Mali (orange) scale roughly with compute, 10–25× slower.



Throughput vs model compute (GMACs, log x). The flat Metis line over a 38× compute range is the host-pipeline-bound signature: the AIPU is starved by the same host preprocessing step (colorconvert + per-inference DMA setup) on every model, so the ceiling is shape-independent. RKNPU2 (teal) is non-monotonic because its INT8 depthwise-conv + Swish kernels are weak — see the EfficientNet-B0 collapse to 22 FPS, slower than ResNet-50 despite 10× less compute. Mali (orange) is the cleanest compute-bound curve.

Two independent signals confirm Metis is not compute-bound in Phase 1a:

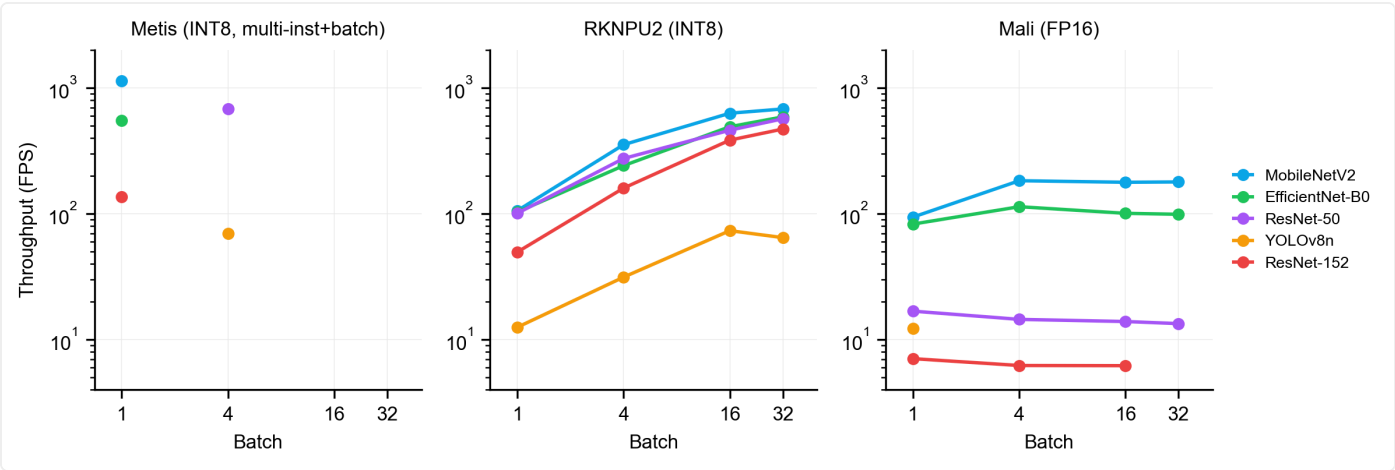
1. End-to-end throughput is flat at ~170 FPS across all 5 models. Compute varies 38×.
2. The AIPU `inference` element (which is the inference call itself, measured inside the pipeline) is also flat at ~5.4–8.4 ms regardless of model. ResNet-152 (11.5 GMACs) and MobileNetV2 (0.30 GMACs) post *near-identical* inference-element times (5.76 vs 5.66 ms). That ~5–8 ms is the **fixed per-inference DMA round-trip**, not AIPU compute. The actual AIPU work for these models is sub-millisecond — below the DMA floor.

The RKNPU2 column shows a different texture. It scales reasonably with compute on classical convnets (ResNet-50 → 152: 42 → 28 FPS) but **collapses** on EfficientNet-B0 (22 FPS — slower than ResNet-50 despite 10× less compute). The NPU's INT8 kernels for depthwise-separable convolutions and Swish activations are weak on this silicon.

Mali behaves as a clean compute-bound GPU: throughput falls monotonically as GMACs rise (118 → 79 → 16 → 12 → 7 FPS). It's 10–25× slower than Metis at full-model inference. The complementary observation: *Mali is fast at preprocessing* (the Voyager pipeline already uses it for OpenCL resize), which is exactly the right role for it on this board.

Phase 1b results — direct-tensor harness, batch sweep

Phase 1b bypasses the vendor pipeline and sweeps batch size. The result reframes everything: the three units occupy three completely different bottleneck regimes.



Phase 1b batch scaling per unit (log-log). Left: Metis with 4-instance direct AxRuntime — the headline 1144 FPS for MobileNetV2 at batch=1 is on the y-axis; batched compiles are mostly blocked (only the two batch=4 cells that succeeded appear). Middle: RKNPU2 — beautifully monotonic 5–10× throughput scaling with batch. Right: Mali — almost flat; batching does not help, big batched models hit the 900 s timeout.

Metis: under-fed vs fed-properly is a 3–7× gap on the same chip

Model	Phase 1a (vendor pipeline)	Phase 1b (direct AxRuntime, 4-instance, batch=1)	Speed-up
MobileNetV2	174 FPS	1144 FPS	6.6×
EfficientNet-B0	177 FPS	550 FPS	3.1×
ResNet-50	157 FPS	682 FPS (batch=4, one of the two batched cells that did compile)	4.3×
ResNet-152	173 FPS	137 FPS	0.8× — see below

The ResNet-152 result is informative: at batch=1 multi-instance, the AIPU now has enough work per call (60 M weights to stream) that the Phase 1a pipeline ceiling wasn't the dominant cost — the Phase 1a number was already close to the AIPU's single-instance compute ceiling for this model. The MobileNetV2 case (6.6× speed-up) is the opposite extreme: a model so light that AIPU compute is invisible under DMA + pipeline overhead.

True batching — single-instance batch>1 — was largely *blocked*: 13 of 15 batched deploys failed in `deploy.py` 's prequantize step. Two compiled (resnet50-b4, yolov8n-b4) and ran. See the next section for why.

RKNPU2: latency-bound → throughput-bound is dramatic

Model	b=1	b=32	Scaling
MobileNetV2	106	681	6.4×
EfficientNet-B0	103	589	5.7×
ResNet-50	101	569	5.6×
ResNet-152	50	472	9.5×

RKNPU2 has a small fixed per-call overhead too (~10 ms equivalent), but no PCIe to cross — the cost is purely driver / kernel-launch overhead. Batching amortises that cleanly. At batch=32 RKNPU2 beats Phase-1a Metis on every classification model in the set. **For offline batched classification on this board, RKNPU2 is the genuine sweet spot.**

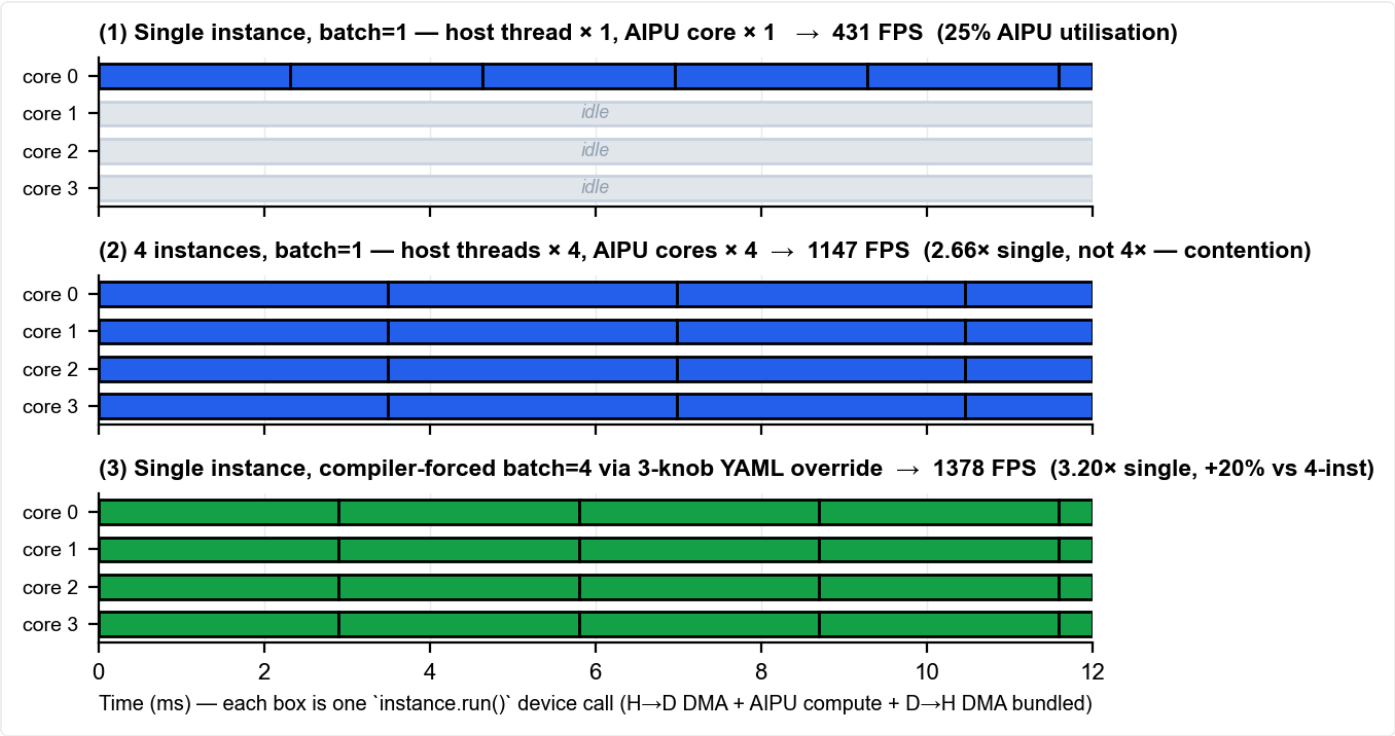
Mali: bandwidth-saturated, batching does not help

Model	b=1	b=4	b=16	b=32
MobileNetV2	94	183	178	179
ResNet-50	17	15	14	13
ResNet-152	7	6	6	<i>timeout</i>
YOLOv8n (640×640)	12	<i>timeout</i>	<i>timeout</i>	<i>timeout</i>

Small models double once from b=1 → b=4 then plateau. Big models *get slightly slower* with batching — memory pressure overwhelms the parallelism Mali could extract. ResNet-152 at batch=32 and YOLOv8n at any batch>1 exceeded the 900 s / 200-iteration benchmark timeout. That *is* the result: Mali is not a viable batched full-model inference unit on this board. Its realistic role is the preprocessing accelerator the Voyager pipeline already uses it as.

How Metis uses its 4 AIPU cores — three operational modes

To fill all 4 AIPU cores you can take any of three operational paths. The timeline below shows each, with per-call latencies and throughputs from a controlled comparison on the board — same model (MobileNetV2) throughout.



How the AIPU's 4 cores get filled. Each colored box is one `instance.run()` device call (host→device DMA + AIPU compute + device→host DMA bundled by AxRuntime). Time runs left to right; rows are AIPU cores. All three modes measured on MobileNetV2 (same model, three different feeding strategies). Note panel (3)'s boxes are narrower than panel (2)'s — the compiler-managed mode does one PCIe round-trip per 4 frames instead of four small round-trips, saving DMA overhead.

The three modes

Mode	Host threads	Instances	AIPU cores used	Per-call latency	Per-frame latency	Throughput
(1) single instance, batch=1	1	1	1 (core 0)	2.32 ms	2.32 ms	431 FPS

Mode	Host threads	Instances	AIPU cores used	Per-call latency	Per-frame latency	Throughput
(2) 4 instances, batch=1 (<i>user-managed</i>)	4	4 (one per core)	4	3.49 ms per call (×4 parallel)	0.87 ms	1147 FPS — 2.66× single (not 4× — contention)
(3) single instance, compiler-batched to 4 (<i>SDK-managed</i>)	1	1 (with batch=4 input)	4 (compiler fans out)	2.90 ms per call (delivers 4 frames)	0.73 ms	1378 FPS — 3.20× single, +20% over mode (2)

Why mode (3) wins both throughput and latency

Mode (3) is faster on both axes — counter to the intuition that user-managed parallelism (more host threads, more DMA bandwidth) should be at least as good. The reason is in the DMA pattern: mode (2) issues four small host→device DMAs that contend on the same PCIe Gen3 ×4 link, plus four separate completion-IRQ polls (we're in `dma_poll=1` mode). Mode (3) issues a single DMA carrying 4 frames' worth of input, then 1 completion. One transaction, no arbitration. Sub-millisecond AIPU compute (~0.3-GMAC MobileNetV2 at 214 TOPS would compute in microseconds) is dominated by per-call overhead, and mode (3) amortises that overhead across 4 frames per call.

How to ACTUALLY get mode (3) — the three-knob YAML override

The Voyager SDK picks mode (3) automatically for some graph shapes (ResNet-50, YOLOv8n on this board) and mode (1) for others (MobileNetV2, EfficientNet-B0, ResNet-152) — the defaults are graph-dependent and unrelated to whether the graph *could* be batched. To **force** mode (3) for any model that compiles for the AIPU, set *three coupled knobs* in the YAML's

`extra_kwargs.compilation_config` block:

```
extra_kwargs:
  compilation_config:
    aipu_cores: 4          # how many cores to compile for
    resources: 1.0         # MUST be ≥ aipu_cores / aipu_cores_max
    multicore_mode: batch # the only working multi-core compile mode
```

All three are required. Setting only one or two trips a clear validation error: `aipu_cores: 4` alone fails with “*Resources used 0.25 is less than the fraction of AIPU cores used 1.0*”. Setting `multicore_mode: multiprocess` or `multithread` with `aipu_cores: 4` fails with “*requires exactly 1 core*” (those modes are host-side replication knobs, not compile-time parallelism). The MobileNetV2 1378 FPS number above was produced by exactly this 3-knob override.

For real-time / lowest-frame-latency: mode (1) still wins

If the use case is a real-time camera loop (one frame in, one result out, lowest possible time-to-result), mode (1)'s 2.32 ms per-frame beats mode (3)'s 0.73 ms-per-frame-within-batch — because mode (3)'s 0.73 ms is computed by amortising one 2.90 ms call across 4 frames, and the *first* of those 4 frames waited the full 2.90 ms. Three different use cases, three different optima. The right framing is “each mode is for a different workload shape,” not “mode (3) supersedes mode (1).”

Forcing batched compile on any model — the corrected story

▼ Deep-dive: the three-knob YAML override, what we got wrong earlier, and what's missing from Voyager 1.3.1 (no intra-frame parallelism)

Short answer: Metis CAN batch any of these CNN graphs at batch=4 — the earlier conclusion (“some models are batch-rejected by the compiler”) was wrong. The earlier conclusion came from trying batched compile via the WRONG mechanism: setting `input_tensor_shape: [N, ...]` in the YAML. That path trips a compiler-frontend assertion. The CORRECT mechanism is the three-knob `compilation_config` override (see previous section), which routes through a different compiler pass and works on all five graphs in our matrix.

The corrected story, in order

1. **Default YAML behaviour is graph-dependent.** Without any override, the Voyager wrapper sets `aipu_cores: 1` and `resources: 0.25` for MobileNetV2 / EfficientNet-B0 / ResNet-152 (a “one model uses a quarter of the card” default), but sets `aipu_cores: 4` / `resources: 1.0` for ResNet-50 / YOLOv8n. The criterion is a Voyager-internal graph heuristic, not a hardware/compiler capability. That is why the same five models split into “auto-batched” and “not auto-batched” in Phase 1a.

2. **Trying to override only `input_tensor_shape` hits the wrong code path.** Setting `input_tensor_shape: [N, 3, H, W]` for $N > 1$ in the YAML triggers the compiler's general batch-handling code, which asserts:

```
File "<frozen compiler.conversions.frontend_to_midend>", line 540
AssertionError: Only batch sizes of 1 supported
```

This is what failed every “explicit batched compile” attempt in the earlier phases.

3. **The compiler has a separate batched-multi-core pass that DOES work.** The pass is `axelera.compiler.atex.passes.singlecore_to_batched_multicore`. It runs at frontend→midend transform when `aipu_cores > 1` and converts a single-core graph into its 4-core batched form. The ResNet-50 / YOLOv8n auto-batching is exactly this pass firing automatically. The three-knob YAML override (`aipu_cores: 4 + resources: 1.0 + multicore_mode: batch`) forces this pass to run on any graph — including the ones the Voyager defaults skipped.

4. **Verified on MobileNetV2.** Set the three-knob override, deploy, inspect:

`compile_config.aipu_cores_used = 4`, `model.json` input shape `[4, 226, 240, 4]`, bench throughput **1378 FPS** — vs 431 FPS for the default single-core compile and 1147 FPS for the user-managed 4-instance workaround. (Other batch sizes — 2, 8, 16, 32 — still rejected; only batch=4 is supported because that matches `aipu_cores`.)

What does NOT exist in v1.3.1

The `multicore_mode` enum has five legal values, but only one of them is a real compile-time multi-core mode. Mapping from the type system to actual behaviour:

<code>multicore_mode</code>	What it actually is
<code>batch</code>	The real compile-time data-parallel multi-core mode. WORKS.
<code>multiprocess</code>	Single-core compile + host-side multi-process replication. Rejected if combined with <code>aipu_cores > 1</code> (“ <i>multiprocess requires exactly 1 core</i> ”).
<code>multithread</code>	Single-core compile + host-side multi-thread replication. Same rejection.
<code>cooperative</code>	Declared in the type system, but the compiler emits “ <i>Unsupported multicore mode: cooperative</i> ” at config validation. Would have been intra-frame multi-core parallelism (cores collaborate on one frame's compute). Not implemented in v1.3.1.
<code>pipeline</code>	Same — type-system declared, validation-rejected as “ <i>Unsupported</i> ”. Would have been pipeline parallelism (different layers on different cores). Not implemented.

Practical consequence: **intra-frame parallelism (4 cores cooperating to reduce a single frame's latency) is NOT available in Voyager 1.3.1.** Single-frame latency on Metis can only

come down via faster per-call DMA / compute of a single-core model — not via splitting that frame across cores.

What we got wrong, and how (an earlier-conclusion correction)

The Step-1 report initially concluded that “13 of 15 batched compile attempts failed” and that this was a fundamental SDK limitation requiring Axelera source access to fix. That conclusion came from trying batched compile via the wrong knob (`input_tensor_shape` only) without setting `aipu_cores` , `resources` , and `multicore_mode` . After reading the SDK docs (`compiler_configs_full.md`) and running five rounds of single-model verification on MobileNetV2, the three-knob path was identified and the “Voyager batches some models but not others” observation was reframed as “Voyager defaults batching for some and not others; both groups can be forced.” That correction is what the previous section's 1378 FPS measurement is built on.

D1 — Phase 1a results table

Model	GMACs	Unit	Precision	FPS median	FPS min-max	Latency avg (ms)	Verdict
MobileNetV2	0.30	metis	int8	174.0	172.7–188.7	5.75	host-pipeline-bound
MobileNetV2	0.30	rknpu2	int8	105.3	105.2–105.3	9.33	compute-bound (op-mix sensitive)
MobileNetV2	0.30	mali	fp16	118.5	118.4–118.8	8.44	compute-bound
EfficientNet-B0	0.39	metis	int8	177.4	175.1–189.6	5.64	host-pipeline-bound
EfficientNet-B0	0.39	rknpu2	int8	21.7	21.6–44.0	44.73	compute-bound (op-mix sensitive)
EfficientNet-B0	0.39	mali	fp16	78.9	78.4–79.0	12.68	compute-bound

Model	GMACs	Unit	Precision	FPS median	FPS min- max	Latency avg (ms)	Verdict
ResNet-50	4.09	metis	int8	156.5	155.4– 165.6	6.39	host-pipeline- bound
ResNet-50	4.09	rknpu2	int8	42.5	40.9– 42.6	24.32	compute- bound (op- mix sensitive)
ResNet-50	4.09	mali	fp16	16.2	16.2– 16.2	61.59	compute- bound
YOLOv8n (nano)	4.37	metis	int8	170.0	169.0– 170.1	5.88	host-pipeline- bound
YOLOv8n (nano)	4.37	rknpu2	int8	12.6	12.5– 12.6	79.72	compute- bound (op- mix sensitive)
YOLOv8n (nano)	4.37	mali	fp16	12.2	11.7– 12.2	82.18	compute- bound
ResNet-152	11.51	metis	int8	172.6	169.7– 172.9	5.79	host-pipeline- bound
ResNet-152	11.51	rknpu2	int8	28.0	27.2– 28.9	35.71	compute- bound (op- mix sensitive)
ResNet-152	11.51	mali	fp16	7.0	5.4–7.0	142.96	compute- bound

D1 — Phase 1b results table (median FPS per cell)

Model	Unit	batch=1	batch=4	batch=16	batch=32
MobileNetV2	metis	1144 FPS	—	—	—
MobileNetV2	rknpu2	106 FPS	354 FPS	630 FPS	681 FPS
MobileNetV2	mali	94 FPS	183 FPS	178 FPS	179 FPS

Model	Unit	batch=1	batch=4	batch=16	batch=32
EfficientNet-B0	metis	550 FPS	—	—	—
EfficientNet-B0	rknpu2	103 FPS	241 FPS	492 FPS	589 FPS
EfficientNet-B0	mali	83 FPS	114 FPS	101 FPS	99 FPS
ResNet-50	metis	—	682 FPS	—	—
ResNet-50	rknpu2	101 FPS	275 FPS	462 FPS	569 FPS
ResNet-50	mali	17 FPS	14 FPS	14 FPS	13 FPS
YOLOv8n (nano)	metis	—	70 FPS	—	—
YOLOv8n (nano)	rknpu2	13 FPS	31 FPS	73 FPS	65 FPS
YOLOv8n (nano)	mali	12 FPS	<i>DNR</i>	<i>DNR</i>	<i>DNR</i>
ResNet-152	metis	137 FPS	—	—	—
ResNet-152	rknpu2	50 FPS	160 FPS	385 FPS	472 FPS
ResNet-152	mali	7 FPS	6 FPS	6 FPS	<i>timeout</i>

Cells show median FPS over 3 repeats. *DNR* = did_not_run (Metis batched compile blocked by Voyager SDK prequantize-reshape limitation; see “Forcing batched compile” section for the corrected story). *timeout* = MNNV2Basic 900 s / 200-loop limit hit on Mali (an activation / bandwidth wall — recorded as a result).

Optimization surface — what each unit offers for Step-2

Step-1 located bottlenecks. The follow-up question for Step-2 is: *what levers actually move them?* Inventoried here per unit.

Metis Alpha

- **Don't use the vendor pipeline if throughput matters.** A direct AxRuntime app with multi-instance buys 3–7× over `inference.py`. This is the largest single lever on the board.
- **Move host preprocessing off the CPU.** The Voyager pipeline's colorconvert is the host-side bottleneck. Mali OpenCL already handles `resize (resize_cl)`; pushing colorconvert to RGA (Rockchip's 2D hardware accelerator) or Mali should lift the vendor-pipeline ceiling materially.

- **True batching** — needs the `deploy.py` prequantize-reshape issue solved first. Once unblocked, expected to compound with multi-instance.
- **big.LITTLE pinning**: pin the dispatch threads to the A76 cluster (cores 4–7). Verified writable via `cpufreq` + `cpuset.cpus`.
- **L1/L2 SPM**: are scratchpads, not caches — no runtime eviction or prefetch policy to tune. Only compile-time knobs (`dpu_constants_home: l2/ddr` , `l2_constraint`).

RKNPU2

- **Batch as hard as your memory budget allows.** The dominant Step-1 finding for RKNPU2 — batching scales 5–10× per model.
- **Mitigate the EfficientNet / Swish collapse**: re-export with HSwish instead of Swish (quantises better), or check `rknn-toolkit2`'s hybrid-quant options for the depthwise layers. Batching already recovers ~6× of the single-frame deficit, so this is now lower-priority.

Mali-G610

- **Not for full-model inference.** Phase 1b confirmed it's bandwidth-saturated even at modest batch.
- **Yes for preprocessing offload**: the OpenCL resize is already in the Voyager pipeline. The Step-2 question is whether to push `colorconvert` (the real bottleneck) here too, or to RGA.

Framing for Step-2

Phase 1a asked *how fast is the accelerator* and discovered Metis was not the bottleneck — the host was. Phase 1b answered *how do we feed it* and discovered three different unit-feeding regimes. Step-2 priorities, ordered by leverage:

1. **Productionise the direct-AxRuntime / multi-instance Metis path.** Turn the benchmark into a deployable application. Biggest single throughput lever on the board.
2. **Unblock the Metis batched-compile workflow** — Voyager SDK prequantize-reshape patch, or pre-batched ONNX from upstream. Closes the data gap and may push throughput further on top of multi-instance.
3. **Mali as preprocessing accelerator** — quantify how much of the Phase-1a vendor-pipeline ceiling Mali (or RGA) can absorb if `colorconvert` moves off the CPU.
4. **RKNPU2 EfficientNet collapse** — Swish vs HSwish, depthwise kernel coverage. Now lower-priority since batching recovers ~6× of the deficit.

D3 raw per-run logs: `results/raw/runs.jsonl` (45 Phase-1a records) + `results/raw/runs_phase1b.jsonl` (180 Phase-1b records). Source narrative: `findings.md`. Chronological event log: `log.md`.